

DVD Rental - SQL Practice Questions and Answer Key

PostgreSQL Practice Questions - DVD Rental Database

Schema Reference (dvdrental)

EASY (150 questions)

MEDIUM (30 questions)

HARD (20 questions)

Answer Key - DVD Rental Practice Questions

EASY

MEDIUM

HARD

PostgreSQL Practice Questions - DVD Rental Database

Total: 200 questions - 150 Easy | 30 Medium | 20 Hard

Schema Reference (dvdrental)

Table	Key Columns
actor	actor_id, first_name, last_name, last_update
address	address_id, address, address2, district, city_id, postal_code, phone
category	category_id, name
city	city_id, city, country_id
country	country_id, country
customer	customer_id, store_id, first_name, last_name, email, address_id, activebool, create_date, active
film	film_id, title, description, release_year, language_id, rental_duration, rental_rate, length, replacement_cost, rating, special_features
film_actor	actor_id, film_id
film_category	film_id, category_id
inventory	inventory_id, film_id, store_id
language	language_id, name
payment	payment_id, customer_id, staff_id, rental_id, amount, payment_date
rental	rental_id, rental_date, inventory_id, customer_id, return_date, staff_id
staff	staff_id, first_name, last_name, address_id, email, store_id, active, username
store	store_id, manager_staff_id, address_id

The full answer key follows the questions later in this document. Try each question yourself before checking.

EASY (150 questions)

A. Basic SELECT & Columns (1–15)

- Select all columns from the actor table.
- Select first_name and last_name from actor .
- Select title , release_year , and rating from film .
- Select customer_id , first_name , last_name , email from customer .
- Select all columns from category .
- Select title and length from film .

7. Select `first_name` , `last_name` , `email` from `staff` .
8. Select `payment_id` , `amount` , `payment_date` from `payment` .
9. Select `title` and `rental_rate` from `film` , aliasing `rental_rate` as `price` .
10. Select `first_name` and `last_name` from `actor` , concatenated into one column called `full_name` .
11. Select all columns from `country` .
12. Select `city` and `country_id` from `city` .
13. Select `rental_id` , `rental_date` , `return_date` from `rental` .
14. Select `name` from `language` .
15. Select `address` , `district` , `postal_code` from `address` .

B. WHERE Clause Filtering (16–35)

16. Find all films with a `rating` of `'PG'` .
17. Find all actors with `first_name` equal to `'Nick'` .
18. Find all films where `rental_rate` is greater than `4.00` .
19. Find all customers who are `active` (active = 1).
20. Find all films with `length` greater than 120 minutes.
21. Find all films released in the year `2006` .
22. Find all customers with `store_id` equal to `1` .
23. Find all payments with `amount` greater than `5.00` .
24. Find all films rated `'R'` with `rental_duration` equal to `7` .
25. Find all actors whose `last_name` is `'Davis'` .
26. Find all films with `replacement_cost` between `10` and `20` .
27. Find all customers whose `email` contains the domain `'sakilacustomer.org'` .
28. Find all films where `rating` is `'PG-13'` or `'R'` .
29. Find all staff members who are active.
30. Find all films with `rental_rate` equal to `0.99` .
31. Find all rentals with a `return_date` that is `NULL` .
32. Find all addresses located in `district` `'California'` .
33. Find all films whose `length` is between `90` and `120` .
34. Find all customers whose `first_name` starts with `'A'` .
35. Find all payments made with `amount` equal to `0` .

C. ORDER BY & LIMIT (36–50)

36. List all films ordered by `title` alphabetically.
37. List all films ordered by `rental_rate` from highest to lowest.
38. List the top 10 longest films by `length` .
39. List all actors ordered by `last_name` , then `first_name` .
40. List the 5 most recent payments by `payment_date` .
41. List all customers ordered by `create_date` ascending.
42. List the 10 cheapest films by `rental_rate` .
43. List all films ordered by `release_year` descending.
44. List the 5 shortest films by `length` .
45. List all cities ordered alphabetically by `city` .
46. List the first 20 rows of the `rental` table ordered by `rental_id` .
47. List the 3 most expensive films by `replacement_cost` .
48. List all staff ordered by `staff_id` .
49. List the 10 most recent rentals by `rental_date` .
50. List all countries ordered alphabetically.

D. DISTINCT (51–60)

51. List all distinct `rating` values in the `film` table.
52. List all distinct `district` values from `address` .
53. List all distinct `rental_duration` values from `film` .
54. List all distinct `store_id` values from `customer` .
55. List all distinct `country_id` values from `city` .

56. List all distinct `staff_id` values that have processed a payment.
57. List all distinct `rental_rate` values from `film`.
58. List all distinct `active` values from `staff`.
59. List all distinct `customer_id` values that appear in `rental`.
50. List all distinct `film_id` values that appear in `inventory`.

E. Aggregate Functions - No Grouping (61–80)

51. Count the total number of films.
52. Count the total number of customers.
53. Count the total number of actors.
54. Find the average `rental_rate` of all films.
55. Find the maximum `length` among all films.
56. Find the minimum `rental_rate` among all films.
57. Find the total (sum) of all `amount` values in `payment`.
58. Find the average `amount` paid per payment.
59. Count the number of films rated `'R'`.
70. Count the number of active customers.
71. Find the maximum `amount` ever paid in a single payment.
72. Find the minimum `amount` ever paid in a single payment.
73. Count how many rentals have a `NULL` `return_date`.
74. Find the average `length` of all films.
75. Count the number of distinct countries in the `country` table.
76. Find the total `replacement_cost` of all films combined.
77. Count the number of rows in the `payment` table.
78. Find the average `rental_duration` across all films.
79. Count the number of stores in the `store` table.
30. Find the maximum `rental_duration` among all films.

F. Pattern Matching & String Functions (81–100)

31. Find all films whose `title` starts with `'A'`.
32. Find all actors whose `last_name` ends with `'son'`.
33. Find all customers whose `email` contains `'smith'` (case-insensitive).
34. Find all films whose `title` contains the word `'Love'`.
35. Convert all actor `first_name` values to uppercase.
36. Convert all film `title` values to lowercase.
37. Find the length (number of characters) of each film `title`.
38. Concatenate `first_name` and `last_name` of customers with a space between them.
39. Find all films whose `title` contains exactly one word (no spaces).
30. Trim any leading/trailing spaces from `address` in the `address` table.
31. Find all actors whose `first_name` has exactly 4 characters.
32. Find all films whose `title` starts with any of `'A'`, `'B'`, or `'C'`.
33. Replace all spaces in film `title` with underscores.
34. Find the first 3 characters of each customer's `last_name`.
35. Find all customers whose `first_name` does NOT start with `'J'`.
36. Find all films whose `description` contains the word `'action'` (case-insensitive).
37. Find all staff whose `username` contains a digit-free pattern (only letters).
38. Extract the domain portion (after `@`) of each customer `email`.
39. Find all actors whose full name (first + last) is longer than 15 characters.
30. Find all films whose `title` ends with `'S'`.

G. Date & Time Functions (101–120)

31. Find all rentals made in the year `2005`.
32. Find all payments made in the month of `May`.
33. Find all rentals made on a `Monday`.
34. Extract just the year from each `payment_date`.

35. Extract just the month from each `rental_date` .
36. Find all customers who joined (created) before `2006-02-01` .
37. Calculate the number of days between `rental_date` and `return_date` for each rental.
38. Find all rentals that have not yet been returned (i.e., still checked out).
39. Find all payments made in the last 7 days of data (relative to the max `payment_date`).
40. Find the earliest `rental_date` in the entire `rental` table.
41. Find the latest `payment_date` in the entire `payment` table.
42. Find all rentals made between `2005-05-01` and `2005-05-31` .
43. Find the day of the week for each `payment_date` .
44. Count how many rentals occurred each year.
45. Find all customers whose `create_date` falls on a weekend.
46. Add 3 days to every `rental_date` and display the result.
47. Find all rentals where the rental happened more than 5 days before the return.
48. Find the current date and current timestamp (using PostgreSQL functions).
49. Find all films' `last_update` values that occurred in `2006` .
50. Calculate how many hours between `rental_date` and `return_date` for the first 10 rentals.

H. NULL Handling (121–130)

21. Find all rentals where `return_date` IS NULL.
22. Find all customers where `email` IS NULL.
23. Find all addresses where `address2` IS NULL.
24. Find all rentals where `return_date` IS NOT NULL.
25. Replace `NULL` values in `address2` with the text `'N/A'` .
26. Count how many `address2` values are NULL versus not NULL.
27. Find all films where `description` IS NOT NULL.
28. Use `COALESCE` to show `'No Description'` when a film's `description` is NULL.
29. Find all staff where `picture` IS NULL.
30. Count the number of NULL `return_date` values in `rental` .

I. Table-Specific Basics (131–145)

31. List all films in the `'Action'` -sounding titles (title contains `'War'`).
32. List all customers who live in `store_id = 2` .
33. List all inventory items belonging to `store_id = 1` .
34. List all languages available in the `language` table.
35. List all categories available in the `category` table.
36. List the `manager_staff_id` for each store.
37. List all films with `special_features` containing `'Trailers'` .
38. List all customers whose `activebool` is `false` .
39. List all payments processed by `staff_id = 1` .
40. List all rentals handled by `staff_id = 2` .
41. List all addresses without a `phone` number (empty string).
42. List all films where `language_id = 1` .
43. List all actors with `actor_id` less than `10` .
44. List all customers with `customer_id` between `1` and `20` .
45. List all films with `rating` of `'NC-17'` .

J. Computed Columns & Arithmetic (146–150)

46. Show each film's `rental_rate` multiplied by `2` as `double_rate` .
 47. Show each film's `replacement_cost` minus `rental_rate` as `cost_difference` .
 48. Show each payment's `amount` rounded to the nearest whole number.
 49. Show each film's `length` converted from minutes to hours (as a decimal).
 50. Show a new column that flags films as `'Long'` if `length > 120` , otherwise `'Short'` (using CASE).
-

MEDIUM (30 questions)

A. Two-Table Joins (151–158)

51. List each film's `title` along with its `category` name.
52. List each customer's `first_name`, `last_name`, and the `city` they live in.
53. List each rental's `rental_id` along with the `title` of the film rented.
54. List each payment's `amount` along with the `first_name` and `last_name` of the customer who paid.
55. List each actor's name along with the titles of films they acted in.
56. List each film's `title` along with its `language` name.
57. List each staff member's name along with the `store_id` they work at.
58. List each address along with its `city` name.

B. Multi-Table Joins (159–164)

59. List each customer's name along with the `city` and `country` they live in.
60. List each rental's `rental_id`, the `customer` name, and the `film title` involved.
61. List each payment's `amount`, the `customer` name, and the `staff` name who processed it.
62. List each film's `title` along with all `category` names it belongs to (via `film_category`).
63. List each actor's full name along with the `title` and `category` of each film they appeared in.
64. List each store's `store_id` along with the `city` where the store is located.

C. GROUP BY & HAVING (165–172)

65. Count the number of films in each `category`.
66. Find the total `amount` paid by each customer.
67. Find the average `rental_rate` for each film `rating`.
68. Count the number of rentals handled by each `staff_id`.
69. Find the number of films for each `language_id`.
70. Find categories that have more than 60 films (use HAVING).
71. Find customers who have paid more than `$100` in total (use HAVING).
72. Find the average payment `amount` per staff member, only showing staff whose average exceeds `$4`.

D. Subqueries (173–177)

73. Find all films whose `rental_rate` is higher than the average `rental_rate` of all films.
74. Find all customers who have never made a rental (using NOT IN or NOT EXISTS).
75. Find all films that are NOT present in any store's `inventory`.
76. Find the customer(s) who made the single highest payment (using a subquery on MAX).
77. Find all actors who appeared in the film `'Academy Dinosaur'` (using a subquery).

E. CASE Expressions & Basic Set Logic (178–180)

78. Categorize each film as `'Budget'` (`rental_rate` < 1), `'Standard'` (1–3), or `'Premium'` (> 3) using CASE.
79. List all `city` names that exist in the `city` table but return no customers living there (LEFT JOIN + NULL check).
80. List all distinct `customer_id` values that appear in either the `rental` table or the `payment` table (UNION).

HARD (20 questions)

A. Window Functions (181–185)

81. For each film, show its `rental_rate` along with the average `rental_rate` of its `category`, using a window function (no GROUP BY / no self-join).
82. Rank films within each `category` by `rental_rate` descending using `RANK()`.
83. For each customer, show their payments along with a running total of `amount` ordered by `payment_date` (using `SUM() OVER`).
84. Find the top 3 highest-grossing films (by total rental revenue) within each `category` using `ROW_NUMBER()` partitioned by category.
85. For each customer, calculate the number of days since their previous rental using `LAG()`.

B. CTEs (186–189)

86. Using a CTE, find the total revenue generated by each store, then return only stores with revenue above the overall average store revenue.

- 37. Using a CTE, find the top 5 customers by total spend, then join that result back to `customer` to show their email addresses.
- 38. Using a recursive CTE, generate a sequence of numbers from 1 to 10 (demonstrating recursive CTE mechanics).
- 39. Using a CTE, calculate each actor's total number of films, then return only actors in the top 10% by film count.

C. Correlated Subqueries (190–193)

- 30. For each customer, find their most recent rental date using a correlated subquery.
- 31. Find all films whose `rental_rate` is above the average `rental_rate` within their own `category` (correlated subquery).
- 32. For each category, find the single most rented film (by rental count) using a correlated subquery.
- 33. Find customers whose total payments exceed the average total payments of all customers (correlated subquery, not HAVING).

D. Self-Joins & Complex Aggregation (194–196)

- 34. Find pairs of actors who have both appeared in the same film together (self-join on `film_actor`, avoiding duplicate pairs).
- 35. Find all customers who rented the exact same film more than once.
- 36. Calculate the month-over-month change in total rental revenue (using a self-join or window function on a monthly aggregate).

E. Advanced Multi-Step Analytics (197–200)

- 37. For each store, find its best-performing month by revenue (store, month, and revenue), using CTEs/window functions.
- 38. Identify the "long-tail" films - those rented fewer than 5 times total - and show their `title`, `category`, and rental count.
- 39. Build a full customer lifetime value report: for each customer, show total spend, number of rentals, average spend per rental, and their rank among all customers by total spend.
- 30. Find the film in each language that has generated the most total revenue, along with that revenue amount, using a single query (CTE + window function).

Tip: Work through Easy questions first to get comfortable with `SELECT`, `WHERE`, `ORDER BY`, and aggregates before moving to Medium (joins/grouping) and Hard (window functions, CTEs, correlated subqueries). -e

Answer Key - DVD Rental Practice Questions

Solutions correspond to the numbered questions in `dvdrental_practice_questions.md`. Many questions have more than one valid solution - these are one correct approach, not the only one.

EASY

```
1. SELECT * FROM actor; 2. SELECT first_name, last_name FROM actor; 3. SELECT title, release_year, rating FROM film; 4. SELECT
customer_id, first_name, last_name, email FROM customer; 5. SELECT * FROM category; 6. SELECT title, length FROM film; 7. SELECT
first_name, last_name, email FROM staff; 8. SELECT payment_id, amount, payment_date FROM payment; 9. SELECT title, rental_rate AS
price FROM film; 10. SELECT first_name || ' ' || last_name AS full_name FROM actor; 11. SELECT * FROM country; 12. SELECT city,
country_id FROM city; 13. SELECT rental_id, rental_date, return_date FROM rental; 14. SELECT name FROM language; 15. SELECT address,
district, postal_code FROM address;

16. SELECT * FROM film WHERE rating = 'PG'; 17. SELECT * FROM actor WHERE first_name = 'Nick'; 18. SELECT * FROM film WHERE
rental_rate > 4.00; 19. SELECT * FROM customer WHERE active = 1; 20. SELECT * FROM film WHERE length > 120; 21. SELECT * FROM film
WHERE release_year = 2006; 22. SELECT * FROM customer WHERE store_id = 1; 23. SELECT * FROM payment WHERE amount > 5.00; 24. SELECT *
FROM film WHERE rating = 'R' AND rental_duration = 7; 25. SELECT * FROM actor WHERE last_name = 'Davis'; 26. SELECT * FROM film WHERE
replacement_cost BETWEEN 10 AND 20; 27. SELECT * FROM customer WHERE email LIKE '%sakilacustomer.org%'; 28. SELECT * FROM film WHERE
rating IN ('PG-13', 'R'); 29. SELECT * FROM staff WHERE active = true; 30. SELECT * FROM film WHERE rental_rate = 0.99; 31. SELECT *
FROM rental WHERE return_date IS NULL; 32. SELECT * FROM address WHERE district = 'California'; 33. SELECT * FROM film WHERE length
BETWEEN 90 AND 120; 34. SELECT * FROM customer WHERE first_name LIKE 'A%'; 35. SELECT * FROM payment WHERE amount = 0;

36. SELECT * FROM film ORDER BY title ASC; 37. SELECT * FROM film ORDER BY rental_rate DESC; 38. SELECT * FROM film ORDER BY length
DESC LIMIT 10; 39. SELECT * FROM actor ORDER BY last_name, first_name; 40. SELECT * FROM payment ORDER BY payment_date DESC LIMIT 5;
41. SELECT * FROM customer ORDER BY create_date ASC; 42. SELECT * FROM film ORDER BY rental_rate ASC LIMIT 10; 43. SELECT * FROM film
ORDER BY release_year DESC; 44. SELECT * FROM film ORDER BY length ASC LIMIT 5; 45. SELECT * FROM city ORDER BY city ASC; 46. SELECT
* FROM rental ORDER BY rental_id LIMIT 20; 47. SELECT * FROM film ORDER BY replacement_cost DESC LIMIT 3; 48. SELECT * FROM staff
ORDER BY staff_id; 49. SELECT * FROM rental ORDER BY rental_date DESC LIMIT 10; 50. SELECT * FROM country ORDER BY country ASC;

51. SELECT DISTINCT rating FROM film; 52. SELECT DISTINCT district FROM address; 53. SELECT DISTINCT rental_duration FROM film; 54.
SELECT DISTINCT store_id FROM customer; 55. SELECT DISTINCT country_id FROM city; 56. SELECT DISTINCT staff_id FROM payment; 57.
SELECT DISTINCT rental_rate FROM film; 58. SELECT DISTINCT active FROM staff; 59. SELECT DISTINCT customer_id FROM rental; 60.
SELECT DISTINCT film_id FROM inventory;

61. SELECT COUNT(*) FROM film; 62. SELECT COUNT(*) FROM customer; 63. SELECT COUNT(*) FROM actor; 64. SELECT AVG(rental_rate) FROM
film; 65. SELECT MAX(length) FROM film; 66. SELECT MIN(rental_rate) FROM film; 67. SELECT SUM(amount) FROM payment; 68. SELECT
AVG(amount) FROM payment; 69. SELECT COUNT(*) FROM film WHERE rating = 'R'; 70. SELECT COUNT(*) FROM customer WHERE active = 1; 71.
SELECT MAX(amount) FROM payment; 72. SELECT MIN(amount) FROM payment; 73. SELECT COUNT(*) FROM rental WHERE return_date IS NULL; 74.
SELECT AVG(length) FROM film; 75. SELECT COUNT(DISTINCT country_id) FROM country; 76. SELECT SUM(replacement_cost) FROM film; 77.
SELECT COUNT(*) FROM payment; 78. SELECT AVG(rental_duration) FROM film; 79. SELECT COUNT(*) FROM store; 80. SELECT
MAX(rental_duration) FROM film;

81. SELECT * FROM film WHERE title LIKE 'A%'; 82. SELECT * FROM actor WHERE last_name LIKE '%son'; 83. SELECT * FROM customer WHERE
email ILIKE '%smith%'; 84. SELECT * FROM film WHERE title LIKE '%Love%'; 85. SELECT UPPER(first_name) FROM actor; 86. SELECT
LOWER(title) FROM film; 87. SELECT title, LENGTH(title) FROM film; 88. SELECT first_name || ' ' || last_name FROM customer; 89.
SELECT * FROM film WHERE title NOT LIKE '% %'; 90. SELECT TRIM(address) FROM address; 91. SELECT * FROM actor WHERE
LENGTH(first_name) = 4; 92. SELECT * FROM film WHERE title LIKE 'A%' OR title LIKE 'B%' OR title LIKE 'C%'; 93. SELECT REPLACE(title,
' ', '_') FROM film; 94. SELECT LEFT(last_name, 3) FROM customer; 95. SELECT * FROM customer WHERE first_name NOT LIKE 'J%'; 96.
SELECT * FROM film WHERE description ILIKE '%action%'; 97. SELECT * FROM staff WHERE username ~ '^([A-Za-z]+)$'; 98. SELECT
SPLIT_PART(email, '@', 2) FROM customer; 99. SELECT * FROM actor WHERE LENGTH(first_name || ' ' || last_name) > 15; 100. SELECT *
FROM film WHERE title LIKE '%S';

101. SELECT * FROM rental WHERE EXTRACT(YEAR FROM rental_date) = 2005; 102. SELECT * FROM payment WHERE EXTRACT(MONTH FROM
payment_date) = 5; 103. SELECT * FROM rental WHERE EXTRACT(DOW FROM rental_date) = 1; 104. SELECT payment_id, EXTRACT(YEAR FROM
payment_date) AS pay_year FROM payment; 105. SELECT rental_id, EXTRACT(MONTH FROM rental_date) AS rent_month FROM rental; 106. SELECT
* FROM customer WHERE create_date < '2006-02-01'; 107. SELECT rental_id, return_date - rental_date AS days_rented FROM rental; 108.
SELECT * FROM rental WHERE return_date IS NULL; 109. SELECT * FROM payment WHERE payment_date >= (SELECT MAX(payment_date) - INTERVAL
'7 days' FROM payment); 110. SELECT MIN(rental_date) FROM rental; 111. SELECT MAX(payment_date) FROM payment; 112. SELECT * FROM
rental WHERE rental_date BETWEEN '2005-05-01' AND '2005-05-31'; 113. SELECT payment_id, TO_CHAR(payment_date, 'Day') AS day_name FROM
payment; 114. SELECT EXTRACT(YEAR FROM rental_date) AS yr, COUNT(*) FROM rental GROUP BY yr ORDER BY yr; 115. SELECT * FROM customer
WHERE EXTRACT(DOW FROM create_date) IN (0,6); 116. SELECT rental_id, rental_date + INTERVAL '3 days' AS new_date FROM rental; 117.
SELECT * FROM rental WHERE return_date - rental_date > INTERVAL '5 days'; 118. SELECT CURRENT_DATE, CURRENT_TIMESTAMP; 119. SELECT *
FROM film WHERE EXTRACT(YEAR FROM last_update) = 2006; 120. SELECT rental_id, EXTRACT(EPOCH FROM (return_date - rental_date))/3600 AS
hours FROM rental ORDER BY rental_id LIMIT 10;

121. SELECT * FROM rental WHERE return_date IS NULL; 122. SELECT * FROM customer WHERE email IS NULL; 123. SELECT * FROM address
WHERE address2 IS NULL; 124. SELECT * FROM rental WHERE return_date IS NOT NULL; 125. SELECT COALESCE(address2, 'N/A') FROM address;
126. SELECT COUNT(*) FILTER (WHERE address2 IS NULL) AS null_count, COUNT(*) FILTER (WHERE address2 IS NOT NULL) AS not_null_count
FROM address; 127. SELECT * FROM film WHERE description IS NOT NULL; 128. SELECT title, COALESCE(description, 'No Description') FROM
film; 129. SELECT * FROM staff WHERE picture IS NULL; 130. SELECT COUNT(*) FROM rental WHERE return_date IS NULL;
```

131. SELECT * FROM film WHERE title LIKE '%War%'; 132. SELECT * FROM customer WHERE store_id = 2; 133. SELECT * FROM inventory WHERE store_id = 1; 134. SELECT * FROM language; 135. SELECT * FROM category; 136. SELECT store_id, manager_staff_id FROM store; 137. SELECT * FROM film WHERE 'Trailers' = ANY(special_features); 138. SELECT * FROM customer WHERE activebool = false; 139. SELECT * FROM payment WHERE staff_id = 1; 140. SELECT * FROM rental WHERE staff_id = 2; 141. SELECT * FROM address WHERE phone = ''; 142. SELECT * FROM film WHERE language_id = 1; 143. SELECT * FROM actor WHERE actor_id < 10; 144. SELECT * FROM customer WHERE customer_id BETWEEN 1 AND 20; 145. SELECT * FROM film WHERE rating = 'NC-17';

146. SELECT title, rental_rate * 2 AS double_rate FROM film; 147. SELECT title, replacement_cost - rental_rate AS cost_difference FROM film; 148. SELECT payment_id, ROUND(amount) FROM payment; 149. SELECT title, length / 60.0 AS length_hours FROM film; 150. SELECT title, length, CASE WHEN length > 120 THEN 'Long' ELSE 'Short' END AS length_flag FROM film;

MEDIUM

151.

```
SELECT f.title, c.name AS category
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id;
```

152.

```
SELECT cu.first_name, cu.last_name, ci.city
FROM customer cu
JOIN address a ON cu.address_id = a.address_id
JOIN city ci ON a.city_id = ci.city_id;
```

153.

```
SELECT r.rental_id, f.title
FROM rental r
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id;
```

154.

```
SELECT p.amount, c.first_name, c.last_name
FROM payment p
JOIN customer c ON p.customer_id = c.customer_id;
```

155.

```
SELECT a.first_name, a.last_name, f.title
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
JOIN film f ON fa.film_id = f.film_id;
```

156.

```
SELECT f.title, l.name AS language
FROM film f
JOIN language l ON f.language_id = l.language_id;
```

157. SELECT first_name, last_name, store_id FROM staff; (store_id is directly on staff - no join required)

158.

```
SELECT a.address, ci.city
FROM address a
JOIN city ci ON a.city_id = ci.city_id;
```

159.

```
SELECT cu.first_name, cu.last_name, ci.city, co.country
FROM customer cu
JOIN address a ON cu.address_id = a.address_id
```

```
JOIN city ci ON a.city_id = ci.city_id
JOIN country co ON ci.country_id = co.country_id;
```

160.

```
SELECT r.rental_id, cu.first_name, cu.last_name, f.title
FROM rental r
JOIN customer cu ON r.customer_id = cu.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id;
```

161.

```
SELECT p.amount, cu.first_name AS cust_first, cu.last_name AS cust_last,
       s.first_name AS staff_first, s.last_name AS staff_last
FROM payment p
JOIN customer cu ON p.customer_id = cu.customer_id
JOIN staff s ON p.staff_id = s.staff_id;
```

162.

```
SELECT f.title, STRING_AGG(c.name, ', ') AS categories
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY f.title;
```

163.

```
SELECT a.first_name, a.last_name, f.title, c.name AS category
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
JOIN film f ON fa.film_id = f.film_id
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id;
```

164.

```
SELECT s.store_id, ci.city
FROM store s
JOIN address a ON s.address_id = a.address_id
JOIN city ci ON a.city_id = ci.city_id;
```

165.

```
SELECT c.name, COUNT(*) AS film_count
FROM film_category fc
JOIN category c ON fc.category_id = c.category_id
GROUP BY c.name;
```

166.

```
SELECT customer_id, SUM(amount) AS total_paid
FROM payment
GROUP BY customer_id;
```

167.

```
SELECT rating, AVG(rental_rate) AS avg_rate
FROM film
GROUP BY rating;
```

168. `SELECT staff_id, COUNT(*) FROM rental GROUP BY staff_id;`

169. `SELECT language_id, COUNT(*) FROM film GROUP BY language_id;`

170.

```

SELECT c.name, COUNT(*) AS film_count
FROM film_category fc
JOIN category c ON fc.category_id = c.category_id
GROUP BY c.name
HAVING COUNT(*) > 60;

```

171.

```

SELECT customer_id, SUM(amount) AS total_paid
FROM payment
GROUP BY customer_id
HAVING SUM(amount) > 100;

```

172.

```

SELECT staff_id, AVG(amount) AS avg_amount
FROM payment
GROUP BY staff_id
HAVING AVG(amount) > 4;

```

173.

```

SELECT * FROM film
WHERE rental_rate > (SELECT AVG(rental_rate) FROM film);

```

174.

```

SELECT * FROM customer
WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM rental);

```

175.

```

SELECT * FROM film
WHERE film_id NOT IN (SELECT DISTINCT film_id FROM inventory);

```

176.

```

SELECT * FROM payment
WHERE amount = (SELECT MAX(amount) FROM payment);

```

177.

```

SELECT a.* FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
WHERE fa.film_id = (SELECT film_id FROM film WHERE title = 'Academy Dinosaur');

```

178.

```

SELECT title, rental_rate,
CASE
    WHEN rental_rate < 1 THEN 'Budget'
    WHEN rental_rate BETWEEN 1 AND 3 THEN 'Standard'
    ELSE 'Premium'
END AS price_tier
FROM film;

```

179.

```

SELECT ci.city
FROM city ci
LEFT JOIN address a ON ci.city_id = a.city_id
LEFT JOIN customer cu ON a.address_id = cu.address_id
WHERE cu.customer_id IS NULL;

```

180.

```

SELECT customer_id FROM rental
UNION
SELECT customer_id FROM payment;

```

HARD

181.

```

SELECT f.title, f.rental_rate, c.name AS category,
       AVG(f.rental_rate) OVER (PARTITION BY c.name) AS category_avg_rate
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id;

```

182.

```

SELECT f.title, c.name AS category, f.rental_rate,
       RANK() OVER (PARTITION BY c.name ORDER BY f.rental_rate DESC) AS rate_rank
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id;

```

183.

```

SELECT customer_id, payment_date, amount,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY payment_date) AS running_total
FROM payment;

```

184.

```

WITH film_revenue AS (
  SELECT f.film_id, f.title, c.name AS category, SUM(p.amount) AS revenue
  FROM payment p
  JOIN rental r ON p.rental_id = r.rental_id
  JOIN inventory i ON r.inventory_id = i.inventory_id
  JOIN film f ON i.film_id = f.film_id
  JOIN film_category fc ON f.film_id = fc.film_id
  JOIN category c ON fc.category_id = c.category_id
  GROUP BY f.film_id, f.title, c.name
),
ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
  FROM film_revenue
)
SELECT category, title, revenue FROM ranked WHERE rn <= 3;

```

185.

```

SELECT customer_id, rental_date,
       rental_date - LAG(rental_date) OVER (PARTITION BY customer_id ORDER BY rental_date) AS days_since_prev
FROM rental;

```

186.

```

WITH store_revenue AS (
  SELECT s.store_id, SUM(p.amount) AS revenue
  FROM payment p
  JOIN staff st ON p.staff_id = st.staff_id
  JOIN store s ON st.store_id = s.store_id
  GROUP BY s.store_id
)
SELECT * FROM store_revenue
WHERE revenue > (SELECT AVG(revenue) FROM store_revenue);

```

187.

```

WITH top_spenders AS (
    SELECT customer_id, SUM(amount) AS total_spend
    FROM payment
    GROUP BY customer_id
    ORDER BY total_spend DESC
    LIMIT 5
)
SELECT c.customer_id, c.email, ts.total_spend
FROM top_spenders ts
JOIN customer c ON ts.customer_id = c.customer_id
ORDER BY ts.total_spend DESC;

```

188.

```

WITH RECURSIVE numbers AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM numbers WHERE n < 10
)
SELECT * FROM numbers;

```

189.

```

WITH actor_counts AS (
    SELECT actor_id, COUNT(*) AS film_count
    FROM film_actor
    GROUP BY actor_id
)
SELECT * FROM actor_counts
WHERE film_count >= (
    SELECT PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY film_count) FROM actor_counts
);

```

190.

```

SELECT c.customer_id,
       (SELECT MAX(r.rental_date) FROM rental r WHERE r.customer_id = c.customer_id) AS last_rental
FROM customer c;

```

191.

```

SELECT f.title, f.rental_rate, c.name AS category
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
WHERE f.rental_rate > (
    SELECT AVG(f2.rental_rate)
    FROM film f2
    JOIN film_category fc2 ON f2.film_id = fc2.film_id
    WHERE fc2.category_id = fc.category_id
);

```

192.

```

SELECT c.name AS category, f.title,
       (SELECT COUNT(*) FROM rental r
        JOIN inventory i ON r.inventory_id = i.inventory_id
        WHERE i.film_id = f.film_id) AS rental_count
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
WHERE f.film_id = (
    SELECT i2.film_id
    FROM inventory i2
    JOIN rental r2 ON r2.inventory_id = i2.inventory_id
    JOIN film_category fc2 ON i2.film_id = fc2.film_id
    WHERE fc2.category_id = fc.category_id
    GROUP BY i2.film_id
    ORDER BY COUNT(*) DESC
);

```

```
    LIMIT 1
);
```

193.

```
SELECT customer_id, SUM(amount) AS total_paid
FROM payment p
GROUP BY customer_id
HAVING SUM(amount) > (
    SELECT AVG(cust_total) FROM (
        SELECT SUM(amount) AS cust_total FROM payment GROUP BY customer_id
    ) sub
);
```

194.

```
SELECT fa1.actor_id AS actor1, fa2.actor_id AS actor2, fa1.film_id
FROM film_actor fa1
JOIN film_actor fa2
    ON fa1.film_id = fa2.film_id
    AND fa1.actor_id < fa2.actor_id;
```

195.

```
SELECT r.customer_id, i.film_id, COUNT(*) AS times_rented
FROM rental r
JOIN inventory i ON r.inventory_id = i.inventory_id
GROUP BY r.customer_id, i.film_id
HAVING COUNT(*) > 1;
```

196.

```
WITH monthly_revenue AS (
    SELECT DATE_TRUNC('month', payment_date) AS month, SUM(amount) AS revenue
    FROM payment
    GROUP BY DATE_TRUNC('month', payment_date)
)
SELECT month, revenue,
    revenue - LAG(revenue) OVER (ORDER BY month) AS change_from_prev_month
FROM monthly_revenue
ORDER BY month;
```

197.

```
WITH store_month_revenue AS (
    SELECT s.store_id, DATE_TRUNC('month', p.payment_date) AS month, SUM(p.amount) AS revenue
    FROM payment p
    JOIN staff st ON p.staff_id = st.staff_id
    JOIN store s ON st.store_id = s.store_id
    GROUP BY s.store_id, DATE_TRUNC('month', p.payment_date)
),
ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY store_id ORDER BY revenue DESC) AS rn
    FROM store_month_revenue
)
SELECT store_id, month, revenue FROM ranked WHERE rn = 1;
```

198.

```
SELECT f.title, c.name AS category, COUNT(r.rental_id) AS rental_count
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
JOIN inventory i ON f.film_id = i.film_id
LEFT JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY f.title, c.name
HAVING COUNT(r.rental_id) < 5
ORDER BY rental_count;
```

199.

```
WITH customer_stats AS (  
    SELECT c.customer_id, c.first_name, c.last_name,  
           SUM(p.amount) AS total_spend,  
           COUNT(p.payment_id) AS num_rentals,  
           SUM(p.amount) / COUNT(p.payment_id) AS avg_spend_per_rental  
    FROM customer c  
    JOIN payment p ON c.customer_id = p.customer_id  
    GROUP BY c.customer_id, c.first_name, c.last_name  
)  
SELECT *, RANK() OVER (ORDER BY total_spend DESC) AS spend_rank  
FROM customer_stats  
ORDER BY spend_rank;
```

200.

```
WITH language_revenue AS (  
    SELECT l.language_id, l.name AS language, f.film_id, f.title, SUM(p.amount) AS revenue  
    FROM payment p  
    JOIN rental r ON p.rental_id = r.rental_id  
    JOIN inventory i ON r.inventory_id = i.inventory_id  
    JOIN film f ON i.film_id = f.film_id  
    JOIN language l ON f.language_id = l.language_id  
    GROUP BY l.language_id, l.name, f.film_id, f.title  
)  
ranked AS (  
    SELECT *, ROW_NUMBER() OVER (PARTITION BY language_id ORDER BY revenue DESC) AS rn  
    FROM language_revenue  
)  
SELECT language, title, revenue FROM ranked WHERE rn = 1;
```